A.7 Programme fortran

Un exemple de programme figure ici dans sa totalité, détaillés par quelques explications supplémentaires.

A.7.1 Partie principale du programme

- Avant tout, le nom et la description du programme sont nécessaires

```
C:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
      program topoeffectsANEA_iran
c
c     Author : Nathalie Vannes
c
c     Computation of topographical effects:
c     1- direct effects (surface gravity)       : delta A
c     2- indirect effets (geoid)                : delta N
c     3- direct effets (gradiometry)            : delta E
c     4- direct effects (satellite gravity)     : delta A
c
c     iran covers an area of 20deg x 15deg
C:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
c     to do before (ONE time)!
c     creation of the 3 input files from the 3 etopo grids 1m, 2m and 5m.
c     format .xyz = 3 columns
c     longitude deg | latitude deg | height m
c      --> write on the terminal :
c     grd2xyz /mnt/home_geo/nvannes/gmt/scripts_nv/etopo/ETOPO1_Ice_g_gmt4.grd
c -R44/64/25/40 > iran_etopo1.xyz
c     grd2xyz /mnt/home_geo/nvannes/gmt/scripts_nv/etopo/ETOPO2v2c_i2_LSB.g98
c -R44/64/25/40 > iran_etopo2.xyz
c     grd2xyz /mnt/home_geo/nvannes/gmt/scripts_nv/etopo/ETOPO5.grd
c -R44/64/25/40 > iran_etopo5.xyz
c
```

- L'étape indispensable est la déclaration des noms et dimensions des variables :

```
C:::::MAIN PROGRAM:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
c
      implicit real*8(a-h,o-z)          ! means that (i-n) are integer
      integer*4 today(3), now(3)        ! dimensions for the computation time
c      TO READ THE INPUT DATA
C      THE DIMENSION HAS TO BE THE NUMBER OF POINTS!!!
c     ** remove the comment on the folowing line for etopo1 **
c      dimension ph(1082101),th(1082101),alt(1082101)     !etopo1
c     and write on the terminal: f95 -o anea1 topoeffectANEA_iran.for
c
c     ** remove the comment on the folowing line for etopo2 **
```

```
c       dimension ph(270000),th(270000),alt(270000)     !etopo2
c      and write on the terminal: f95 -o anea2 topoeffectANEA_iran.for
c
c      ** remove the comment on the folowing line for etopo5 **
       dimension ph(43621),th(43621),alt(43621)        !etopo5
c      and write on the terminal: f95 -o anea5 topoeffectANEA_iran.for
c
c      Then execute the 3 exe : ./anea1 ./anea2 ./anea5
c
```

- Information sur les valeurs constantes $\varrho_0, G, \gamma_Q, R$:

```
       pi=4.d00*datan(1.d00)
       rho=2.67d03             ! density
       gnewt=6.67d-11          ! Newton constant
       rg=6371.d03             ! radius geoid in m
       gamma=9.81d00           ! gravity
c
```

- Une question apparait a l'ecran pour demander si le fichier executable doit calculer avec Etopo 1, 2 ou 5 (cela depend le la dimension déclarée) :

```
c      itopo'x itopo' data : (1,2 or 5)
c
       write(6,*) 'NUMBER OF POINTS MUST BE UPDATED'
       write(6,*) 'Define the grid used :'
       write(6,*) ' 1 ---> ETOPO1'
       write(6,*) ' 2 ---> ETOPO2'
       write(6,*) ' 5 ---> ETOPO5'
       read(5,*) itopo
c
```

- Creation de fichiers où sera noté le temps de calcul :

```
       if(itopo.eq.1) open(21,file='tps_comput1.xyz')
       if(itopo.eq.2) open(21,file='tps_comput2.xyz')
       if(itopo.eq.5) open(21,file='tps_comput5.xyz')
c
c      write the date and time at the BEGIN of the computation :
c      idate and itime Intrinsic functions for f95 (NOT f77)
       call idate(today)    ! today(1)=day, (2)=month, (3)=year
       call itime(now)      ! now(1)=hour, (2)=minute, (3)=second
       write ( *, 1000 )  today(2), today(1), today(3), now
       write ( 21, 1000 )  today(2), today(1), today(3), now
 1000 format ( 'BEGIN: ', i2.2, '/', i2.2, '/', i4.4, '; time ',
     &          i2.2, ':', i2.2, ':', i2.2 )
       write(6,*) 'Computation process...'
```

- Le fichier source est ouvert (open) puis lu (read) grâce à une boucle (do-loop) :

```
c
c     number of points for Etopo 1 , 2 or 5
c
      if(itopo.eq.1) npoints=1082101    !etopo1
      if(itopo.eq.2) npoints=270000     !etopo2
      if(itopo.eq.5) npoints=43621      !etopo5
c
      step=1.d00*dfloat(itopo)/60.d00
      ss=step*pi/180.d00
c
c     open the file which will be created
c     it will contain the datas that will be drawn later with gmt
c     the unit is in the System International :
c     The unit is m.s-2,m,s-2,m.s-2 :
      if(itopo.eq.1) open(81,file='sat_iran_etopo1_SI.xyz')
      if(itopo.eq.2) open(81,file='sat_iran_etopo2_SI.xyz')
      if(itopo.eq.5) open(81,file='sat_iran_etopo5_SI.xyz')
c
c     The unit is mGal,m,Eotvos,mGal :
      if(itopo.eq.1) open(82,file='sat_iran_etopo1_unit.xyz')
      if(itopo.eq.2) open(82,file='sat_iran_etopo2_unit.xyz')
      if(itopo.eq.5) open(82,file='sat_iran_etopo5_unit.xyz')
c
c     open the input file which contains the datas which will be computed :
      if(itopo.eq.1) open(58,file='iran_etopo1.xyz')
      if(itopo.eq.2) open(58,file='iran_etopo2.xyz')
      if(itopo.eq.5) open(58,file='iran_etopo5.xyz')
c
c     read the input file :
c
      do 10 ifile=1,npoints
      read (58,*) dlong,dlat,height
      ph(ifile)=dfloat(dlong)
      th(ifile)=dfloat(dlat)
      alt(ifile)=dfloat(height)
   10 continue
      close(58)
```

- Une première boucle est ouverte, afin d'effectuer un calcul en chaque point un à un (computation point) dont l'altitude $H(\Omega)$ est notée 'ha' dans le programme :

```
c
c     integration over spherical cap :
c
      do 20 iha=1,npoints
      ha=alt(iha)
      zero=0.d00
      if (ha.lt.zero) then
```

```
        ha=zero
      endif
      tha=(90.d00-th(iha))*pi/180.d00   ! co-latitude (rad)
      sirka=th(iha)                     ! latitude    (rad)
      pha=ph(iha)*pi/180.d00            ! longitude   (rad)
c
```

• A l'intérieur de cette boucle est inclue une autre boucle qui permet d'effectuer l'intégration selon un cap sphérique c'est-à-dire en intégrant chaque point de la zone de calcul. Leur altitude est $H(\Omega')$ notée hp :

```
c
      sum1=0.d00
      sum2=0.d00
      sum3=0.d00
      sum4=0.d00
      do 21 ihp=1,npoints
      hp=alt(ihp)
      if (hp.lt.zero) then
        hp=zero
      endif
      thp=(90.d00-th(ihp))*pi/180.d00
      php=ph(ihp)*pi/180.d00
      if(iha.eq.ihp) goto 21
      sth=dsin(thp)
      x=dcos(thp)*dcos(tha)+dsin(thp)*dsin(tha)*dcos(php-pha)
```

• Trois differentes subroutines (NEWTC, DNEWTC, DDNEWTC) sont alors appelées afin d'effectuer les calculs en fonction des paramètres entrés. (Voir les subroutines à la fin du programme)

```
c
c     1- call the subroutine to obtain delta A|P
c     direct topographical effect on surface gravity
      call DNEWTC(rg+ha,x,rg,rg+ha,ha,qdea)
      call DNEWTC(rg+ha,x,rg,rg+hp,hp,qdep)
      qker1=rho*(qdep-qdea)
      sum1=sum1+qker1*sth*ss*ss
c
c     2- call the subroutine for delta V|Pg
c     primary indirect topographical effect on potential
      call NEWTC(rg,x,rg,rg+ha,ha,qiea)
      call NEWTC(rg,x,rg,rg+hp,hp,qiep)
      qker2=rho*(qiep-qiea)
      sum2=sum2+qker2*sth*ss*ss
c
c     3- call the subroutine for delta E|Psat(GOCE)
c     direct topographical effect on gradiometry
```

```
      call DDNEWTC(rg+250.d03,x,rg,rg+ha,ha,gocea)
      call DDNEWTC(rg+250.d03,x,rg,rg+hp,hp,gocep)
      qker3=rho*(gocep-gocea)
      sum3=sum3+qker3*sth*ss*ss
c
c     4- call the subroutine for delta A|Psat(GRACE)
c     direct topographical effect on gravity from satellite measurement
      call DNEWTC(rg+400.d03,x,rg,rg+ha,ha,gracea)
      call DNEWTC(rg+400.d03,x,rg,rg+hp,hp,gracep)
      qker4=rho*(gracep-gracea)
      sum4=sum4+qker4*sth*ss*ss
   21 continue
```

- une fois l'integration finie, on peut sortir de cette boucle "interne" et finir le calcul dans l'autre boucle pour calculer chaque effet topographique, point par point :

```
c
c     Final Values:
c     1- delta A|P : direct topographical effect on gravity (m.s-2 SI)
      val1si=sum1*gnewt
c     unit u = mGal --> 1 m.s-2 = 10^5 mGal
      val1u=sum1*gnewt*1.d05
c
c     2- delta N : primary indirect effect on potential (m SI)
      val2=sum2*gnewt/gamma
      valb=-2.d00*pi*gnewt*rho*ha*ha*(1.d00+
     1       2.d00*ha/rg/3.d00)/gamma
      valsumsi=val2+valb
c
c     3- delta E|Ps(GOCE): direct topographical effect on gradiometry (s-2 SI)
      val3si=sum3*gnewt
c     unit u = Eotvos --> 1 s-2 = 10^9 E
      val3u=sum3*gnewt*1.d09
c
c     4- delta A|Ps(GRACE): direct topographical effect on gravity (m.s-2 SI)
      val4si=sum4*gnewt
c     unit = mGal --> 1 m.s-2 = 10^5 mGal
      val4u=sum4*gnewt*1.d05
c
```

- Il ne reste plus qu'à écrire les valeurs obtenues lignes par lignes (=point par point) à l'interieur de la boucle principale, que l'on peut refermer ensuite (contunue) :

```
c
c     Write the final files
c           - ANEA_iran_etopo[1,2 or 5]_SI.xyz :
      write(81,205) pha*180.d00/pi,sirka,ha,val1si,valsumsi,val3si,val4si
c           - and ANEA_iran_etopo[1,2 or 5]_unit.xyz :
      write(82,205) pha*180.d00/pi,sirka,ha,val1u,valsumsi,val3u,val4u
```

```
  205 format(2f11.4,f10.2,4d30.8)
c
   20 continue
```

- Puis le programme se termine par l'écriture du temps de fin de calcul

```
c
c      write the date and time at the END of the computation :
       call idate(today)    ! today(1)=day, (2)=month, (3)=year
       call itime(now)      ! now(1)=hour, (2)=minute, (3)=second
       write ( *, 1001 )  today(2), today(1), today(3), now
       write ( 21, 1001 )  today(2), today(1), today(3), now
 1001 format ( 'END: ', i2.2, '/', i2.2, '/', i4.4, '; time ',
     &         i2.2, ':', i2.2, ':', i2.2 )
c
       stop
       end
c
```

A.7.2 Les trois subroutines appelées dans le programme

```
C:::::THE 3 SUBROUTINES:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
```

- Subroutine "DNEWTC" for the first derivative of the Newton kernel

```
c.....................................................................
       subroutine DNEWTC(r,x,dol,hor,h,qint)
c
c      The integration kernel for the direct topographical effect
c      on gravity delta A
c
c      Inputs:
c          r ..... radius of the computation point
c          x ..... cos(psi)
c          dol .... lower bound of the Newton integral
c          hor  ... upper bound of the Newton integral
c          h ..... topographical height of the computation/source point
c      Outputs:
c          qint ... integration kernel of residual potential
c
c.....................................................................
       implicit real*8(a-h,o-z)
c
c      kernel for potential of topographical masses : A t
c      --> first derivative
c
       rg=6371.d03            ! radius geoid in m
c
```

```
      t1=dol/r
      t2=hor/r
      d1=1.d00-2.d00*t1*x+t1*t1
      d2=1.d00-2.d00*t2*x+t2*t2
      d1=dsqrt(d1)
      d2=dsqrt(d2)
      pom=dabs((t2-x+d2)/(t1-x+d1))
      pom=(3.d00*x*x-1.d00)*dlog(pom)
      pom1=(x*t1*t1+3.d00*x+(1.d00-6.d00*x*x)*t1)/d1
      pom2=(x*t2*t2+3.d00*x+(1.d00-6.d00*x*x)*t2)/d2
      qint1=r*(pom2-pom1+pom)
c
c     kernel for potential of condensed masses : A c
c
      dl=dsqrt(r*r+rg*rg-2.d00*r*rg*x)
      sigma=h*(1.d00+h/rg+h*h/(3.d00*rg*rg))
      qint2=-rg*rg*sigma*(r-rg*x)/(dl**3)
c
c     kernel for residual potential
c
      qint=qint1-qint2
      return
      end
```

- Subroutine "NEWTC" for the Newton kernel

```
      subroutine NEWTC(r,x,dol,hor,h,qint)
c
c     The integration kernel for the primary indirect topographical
c     effect on potential delta V
c
c     Inputs:
c         r ..... radius of the computation point
c         x ..... cos(psi)
c         dol .... lower bound of the Newton integral
c         hor  ... upper bound of the Newton integral
c         h ..... topographical height of the computation/source point
c     Outputs:
c         qint ... integration kernel of residual potential
c.....
      implicit real*8(a-h,o-z)
c
c     kernel for potential of topographical masses Vt
c
      rg=6371.d03            ! radius geoid in m
c
      t1=dol/r
      t2=hor/r
      d1=1.d00-2.d00*t1*x+t1*t1
```

```
      d2=1.d00-2.d00*t2*x+t2*t2
      d1=dsqrt(d1)
      d2=dsqrt(d2)
      pom=(t2-x+d2)/(t1-x+d1)
      pom=dabs(pom)
      pom=(3.d00*x*x-1.d00)*dlog(pom)
      pom=pom+(t2+3.d00*x)*d2-(t1+3.d00*x)*d1
      qint1=r*r*(pom/2.d00)
c
c     kernel for potential of condensed masses Vc
c
      dl=dsqrt(r*r+rg*rg-2.d00*r*rg*x)
      sigma=h*(1.d00+h/rg+h*h/(3.d00*rg*rg))
      qint2=rg*rg*sigma/dl
c
c     kernel for residual potential delta V
c
      qint=qint1-qint2
      return
      end
```

- Subroutine "DDNEWTC" for the second derivative of the Newton kernel

```
      subroutine DDNEWTC(r,x,dol,hor,h,qint)
c
c     The integration kernel for the second derivative delta E
c
c     Inputs:
c         r ..... radius of the computation point
c         x ..... cos(psi)
c         dol .... lower bound of the Newton integral
c         hor  ... upper bound of the Newton integral
c         h ..... topographical height of the computation/source point
c     Outputs:
c         qint ... integration kernel of residual potential
c
c.....
      implicit real*8(a-h,o-z)
c
c     second derivativ of the kernel for potential of topographical masses Vrrt
c     formula (see personal notes,calculated on 10.01.2011)
c
      rg=6371.d03            ! radius geoid in m
c
      dl1=dsqrt(r*r+dol*dol-2.d00*r*dol*x)
      dl2=dsqrt(r*r+hor*hor-2.d00*r*hor*x)
      pom11=(3.d00*x*x-1.d00)*dlog(dabs(dl1-r*x+dol))
      pom12=3.d00*r*r*r*x+r*r*(dol-12.d00*x*x*dol)
      pom13=2.d00*r*x*(6.d00*x*x+1.d00)*dol*dol
```

```
      pom14=(2.d00-8.d00*x*x)*dol*dol*dol
c
      pom21=(3.d00*x*x-1.d00)*dlog(dabs(dl2-r*x+hor))
      pom22=3.d00*r*r*r*x+r*r*(hor-12.d00*x*x*hor)
      pom23=2.d00*r*x*(6.d00*x*x+1.d00)*hor*hor
      pom24=(2.d00-8.d00*x*x)*hor*hor*hor
c
      pom1=pom11+(pom12+pom13+pom14)/(dl1*dl1*dl1)
      pom2=pom21+(pom22+pom23+pom24)/(dl2*dl2*dl2)
      qint1=pom2-pom1
c
c     second derivativ of kernel for potential of condensed masses Vrr c
c
      dl=dsqrt(r*r+rg*rg-2.d00*r*rg*x)
      sigma=h*(1.d00+h/rg+h*h/(3.d00*rg*rg))
      poma=-1.d00/(dl**3)
      pomb=3.d00*(r*r+rg*rg*x*x-2.d00*x*r*rg)/(dl**5)
      qint2=rg*rg*sigma*(poma+pomb)
c
c     kernel for residual potential delta E
c
      qint=qint1-qint2
c
      return
      end
```

Remarques pour la programmation : être vigilant au respect de la mise en forme du fichier code fortran (lu par colonnes), connaître les formats des données en entrée et sortie, harmoniser les unités, commenter les étapes du programme.